ECSE-6320 Advanced Computer Systems Project 1 Report

Cooper Werner
Rensselaer Polytechnic Institute
wernec6@rpi.edu

October 22, 2026

Contents

1 Introduction **3**

2 Pre-Experiment Hypotheses **3**

3 Vectoriziation Validation **4**
3.1 Serial Method . . . 4
3.1.1 Kernel 1: SAXPY . . . 4
3.1.2 Kernel 2: Vector Multiplication . . . 4
3.1.3 Kernel 3: 3 Element Stencil . . . 5
3.2 Autovectorization Method . . . 5
3.2.1 Kernel 1: SAXPY . . . 5
3.2.2 Kernel 2: Vector Multiplication . . . 5
3.2.3 Kernel 3: 3 Element Stencil . . . 6
3.3 Handwritten AVX512 Method . . . 7
3.3.1 Kernel 1: SAXPY . . . 7
3.3.2 Kernel 2: Vector Multiplication . . . 7
3.3.3 Kernel 3: 3 Element Stencil . . . 7
3.4 Verifying Correctness . . . 8
3.4.1 SAXPY . . . 8
3.4.2 Vector Multiply . . . 8
3.4.3 Three Stencil . . . 8

4 Measurement Discipline **9**
4.1 Experimental Setup . . . 9
4.2 Statistical Treatment . . . 9

5 Results **10**
5.1 Summary Figure . . . 10
5.2 Secondary Figures . . . 11

6 Negative / Failure Case **15**

7 Honest Anomalies **16**

8 Discussion **16**

9 Conclusion **16**

A AI Usage **16**

B Additional Data / Raw Logs **17**

1 Introduction

The goal of this project is to analyze the performance of SIMD Instructions and how their performance is affected by the datasize, size of the vector, memory alignment, data size, and compiler optimization.

To do this I will run multiple experiments across three different kernels:

1. Vector Multiply and Accumulate: $\vec{y} = a \times \vec{x} + \vec{y}$
2. Vector Elementwise Multiplication: $z_i = x_i * y_i$
3. 3-Element 1D Stencil: $y_i = a \times x_{i-1} + b \times x_i + c \times x_{i+1}$

For each kernel I will adjust the size of the vector using different vectoriziation instructions (SSE, AVX, AVX512). Adjust memory alignment, element size (FP32 vs FP64), number of elements provided, and compiler flags for autovectorization.

Performance will be evaluated using the Linux `timespec` to measure execution time of the vector execution while isolating overhead from I/O operations, and verification.

2 Pre-Experiment Hypotheses

Pre-Experiment Hypothesis

Hypothesis:

The compiler's autovectorization will produce code that runs significantly faster than the serial version. Since the compiler doesn't have the same knowledge as my code as I do, I think my SIMD implementation will be at least the same speed as the autovectorized version or a bit faster.

Reasoning:

The compiler will target the largest vectors that are available on my CPU since it will be provided with the `-march=znver4` compiler flag. Although my kernels are fairly simple vector operations, since I have better knowledge of my data layout, I think that my handwritten vectoriziation will perform better than the compiler's vectoriziation.

Predicted quantitative outcome:

Table 1: Predicted SIMD speedup relative to scalar baseline

Vectoriziation Strategy	FP32 Speedup from Serial	FP64 Speedup from Serial
Serial	$1x$	$1x$
Compiler Autovectorization	$10.5x$	$5.2x$
Handwritten Vectoriziation	$11x$	$5.6x$

Pre-Experiment Hypothesis

Hypothesis:

Shifting the memory alignment off of the 64 Byte alignment that is preferred by AVX512 will

significantly affect the performance of the SIMD instructions. The instructions will likely still run, however they will not run efficiently and may even be slower than the scalar counterpart.
Reasoning:

CPUs are most efficient when the data is alligned to the memory boundary since CPUs will fetch memory multiple bytes at a time. If the memory is shifted off the memory alignment boundary it will have to do multiple data fetches to fill the buffers. This will increase the number of pipeline stalls and reduce instruction throuhput.
Predicted quantitative outcome:

Table 2: Predicted SIMD Slowdown relative to SIMD with 0 byte offset

Offset Bytes	FP32 Slowdown	FP64 Slowdown
1	∼0.5×	∼0.25×
2	∼0.5×	∼0.25×
3	∼0.5×	∼0.25×
4	∼1×	∼0.25×
5	∼0.5×	∼0.25×
6	∼0.5×	∼0.25×
7	∼0.5×	∼0.25×

3 Vectoriziation Validation

3.1 Serial Method

To prove that the serial versions of the experiments are actually running serially and that the compiler didn't try to automatically vectorize the kernels, we can look at the disassembly of the binary:

3.1.1 Kernel 1: SAXPY

```
0000000000402380 <kernels_SaxpyF32>:
...
 4023c0:       c5 fa 10 04 c7          vmovss xmm0,DWORD PTR [rdi+rax*8]
 4023c5:       c5 fa 59 04 c6          vmulss xmm0,xmm0,DWORD PTR [rsi+rax*8]
 4023ca:       c5 fa 58 04 c2          vaddss xmm0,xmm0,DWORD PTR [rdx+rax*8]
 4023cf:       c5 fa 11 04 c2          vmovss DWORD PTR [rdx+rax*8],xmm0
...
```

3.1.2 Kernel 2: Vector Multiplication

```
00000000004026f0 <kernels_MultiplyF32>:
...
 402720:       c5 fa 10 04 c7          vmovss xmm0,DWORD PTR [rdi+rax*8]
 402725:       c5 fa 59 04 c6          vmulss xmm0,xmm0,DWORD PTR [rsi+rax*8]
 40272a:       c5 fa 11 04 c2          vmovss DWORD PTR [rdx+rax*8],xmm0
```

```
...
```

3.1.3 Kernel 3: 3 Element Stencil

```
0000000000402a50 <kernels_ThreeStencilF32>:
 ...
 402aad:       c5 fa 10 5f 08          vmovss xmm3,DWORD PTR [rdi+0x8]
 402ab2:       b8 01 00 00 00          mov    eax,0x1
 402ab7:       66 0f 1f 84 00 00 00    nop    WORD PTR [rax+rax*1+0x0]
 402abe:       00 00
 402ac0:       c5 fa 59 6c c7 f8       vmulss xmm5,xmm0,DWORD PTR [rdi+rax*8-0x8]
 402ac6:       c5 f2 59 db             vmulss xmm3,xmm1,xmm3
 402aca:       48 ff c0                inc    rax
 402acd:       c5 d2 58 eb             vaddss xmm5,xmm5,xmm3
 402ad1:       c5 fa 10 1c c7          vmovss xmm3,DWORD PTR [rdi+rax*8]
 402ad6:       c5 ea 59 e3             vmulss xmm4,xmm2,xmm3
 402ada:       c5 da 58 e5             vaddss xmm4,xmm4,xmm5
 402ade:       c5 fa 11 64 c6 f8       vmovss DWORD PTR [rsi+rax*8-0x8],xmm4
...
```

Despite the fact that that each kernel utilizes x86 vector instructions (since the arithmetic instructions are prefixed with 'v'), they are postfixed with 'ss' which is x86's way of designating that these instructions only process one scalar single-precision value. So with this we know that this kernel is running fully serially.

3.2 Autovectorization Method

3.2.1 Kernel 1: SAXPY

```
0000000000402380 <kernels_SaxpyF32>:
 ...
 40240e: 62 f2 7d 48 7f 50 ff  vpermt2ps zmm2,zmm0,ZMMWORD PTR [rax-0x40]
 402415: 62 f2 7d 48 7f 59 ff  vpermt2ps zmm3,zmm0,ZMMWORD PTR [rcx-0x40]
 40241c: 62 f2 7d 48 7f 4e ff  vpermt2ps zmm1,zmm0,ZMMWORD PTR [rsi-0x40]
 402423: 62 f2 6d 48 98 cb     vfmadd132ps zmm1,zmm2,zmm3
 402429: 62 f3 7d 28 19 cb 01  vextractf32x4 xmm3,ymm1,0x1
 402430: c4 e3 79 17 48 88 01  vextractps DWORD PTR [rax-0x78],xmm1,0x1
 402437: c4 e3 79 17 48 90 02  vextractps DWORD PTR [rax-0x70],xmm1,0x2
 40243e: c4 e3 79 17 48 98 03  vextractps DWORD PTR [rax-0x68],xmm1,0x3
 402445: c5 fa 11 48 80        vmovss DWORD PTR [rax-0x80],xmm1
 40244a: c5 fa 11 58 a0        vmovss DWORD PTR [rax-0x60],xmm3
...
```

The actual operation for this kernel is the Fused-Multiply-Add operation which here we can see is being performed using packed signle-precision floating point vector operations.

3.2.2 Kernel 2: Vector Multiplication

```
0000000000403250 <kernels_MultiplyF32>:
```

```
...
 4032b0: 62 b1 7c 48 10 04 07  vmovups zmm0,ZMMWORD PTR [rdi+r8*1]
 4032b7: 62 b1 7c 48 10 14 06  vmovups zmm2,ZMMWORD PTR [rsi+r8*1]
 4032be: 48 83 e8 80           sub    rax,0xffffffffffffff80
 4032c2: 62 b2 75 48 7f 54 06  vpermt2ps zmm2,zmm1,ZMMWORD PTR [rsi+r8*1+0x40]
 4032c9: 01
 4032ca: 62 b2 75 48 7f 44 07  vpermt2ps zmm0,zmm1,ZMMWORD PTR [rdi+r8*1+0x40]
 4032d1: 01
 4032d2: 49 83 e8 80           sub    r8,0xffffffffffffff80
 4032d6: 62 f1 7c 48 59 c2     vmulps zmm0,zmm0,zmm2
 4032dc: 62 f3 7d 28 19 c3 01  vextractf32x4 xmm3,ymm0,0x1
 4032e3: c4 e3 79 17 40 88 01  vextractps DWORD PTR [rax-0x78],xmm0,0x1
 4032ea: c4 e3 79 17 40 90 02  vextractps DWORD PTR [rax-0x70],xmm0,0x2
 4032f1: c4 e3 79 17 40 98 03  vextractps DWORD PTR [rax-0x68],xmm0,0x3
 4032f8: c5 fa 11 40 80        vmovss DWORD PTR [rax-0x80],xmm0
 4032fd: c5 fa 11 58 a0        vmovss DWORD PTR [rax-0x60],xmm3
...
```

3.2.3 Kernel 3: 3 Element Stencil

```
0000000000403b10 <kernels_ThreeStencilF32>:
 ...
 403be0: 62 f1 7c 48 10 aa 08  vmovups zmm5,ZMMWORD PTR [rdx+0x8]
 403be7: 00 00 00
 403bea: 62 f1 7c 48 10 22     vmovups zmm4,ZMMWORD PTR [rdx]
 403bf0: 62 f1 7c 48 10 9a 10  vmovups zmm3,ZMMWORD PTR [rdx+0x10]
 403bf7: 00 00 00
 403bfa: 48 83 ea 80           sub    rdx,0xffffffffffffff80
 403bfe: 48 83 e8 80           sub    rax,0xffffffffffffff80
 403c02: 62 f2 7d 48 7f aa c8  vpermt2ps zmm5,zmm0,ZMMWORD PTR [rdx-0x38]
 403c09: ff ff ff
 403c0c: 62 f2 7d 48 7f 62 ff  vpermt2ps zmm4,zmm0,ZMMWORD PTR [rdx-0x40]
 403c13: 62 f2 7d 48 7f 9a d0  vpermt2ps zmm3,zmm0,ZMMWORD PTR [rdx-0x30]
 403c1a: ff ff ff
 403c1d: 62 d1 54 48 59 eb     vmulps zmm5,zmm5,zmm11
 403c23: 62 d2 55 48 98 e4     vfmadd132ps zmm4,zmm5,zmm12
 403c29: 62 d2 5d 48 98 da     vfmadd132ps zmm3,zmm4,zmm10
 403c2f: 62 f3 7d 28 19 dd 01  vextractf32x4 xmm5,ymm3,0x1
 403c36: c4 e3 79 17 58 88 01  vextractps DWORD PTR [rax-0x78],xmm3,0x1
 403c3d: c4 e3 79 17 58 90 02  vextractps DWORD PTR [rax-0x70],xmm3,0x2
 403c44: c4 e3 79 17 58 98 03  vextractps DWORD PTR [rax-0x68],xmm3,0x3
...
```

When running with GCC autovectorization enabled, all three kernels generated vectorized operations with the 'ps' suffix which x86 uses to denote vector operations on multiple single precision floats at once.

3.3 Handwritten AVX512 Method

3.3.1 Kernel 1: SAXPY

```
0000000000402380 <kernels_SaxpyF32>:
 ...
 4023e4: 62 f2 7d 4a 92 0c 81  vgatherdps zmm1{k2},DWORD PTR [rcx+zmm0*4]
 4023eb: 49 8d 0c 02           lea    rcx,[r10+rax*1]
 4023ef: 62 f2 7d 4b 92 1c 81  vgatherdps zmm3{k3},DWORD PTR [rcx+zmm0*4]
 4023f6: 48 8d 0c 07           lea    rcx,[rdi+rax*1]
 4023fa: 4c 01 d8              add    rax,r11
 4023fd: 62 f2 7d 4c 92 14 81  vgatherdps zmm2{k4},DWORD PTR [rcx+zmm0*4]
 402404: 62 f2 6d 48 98 cb     vfmadd132ps zmm1,zmm2,zmm3
 40240a: 62 f2 7d 4d a2 0c 81  vscatterdps DWORD PTR [rcx+zmm0*4]{k5},zmm1
...
```

To handle data of different strides, I used the `vgatherdps` and `vscatterdps` instructions to gather and scatter data even when seperated by up to 64 bytes. The Fused-Multiply-Add instruction does the arithmetic.

3.3.2 Kernel 2: Vector Multiplication

```
0000000000402710 <kernels_MultiplyF32>:
 ...
 40275c: 62 f2 7d 4a 92 14 8f  vgatherdps zmm2{k2},DWORD PTR [rdi+zmm1*4]
 402763: 49 8d 3c 02           lea    rdi,[r10+rax*1]
 402767: c5 f8 90 e1           kmovw  k4,k1
 40276b: 62 f2 7d 4b 92 04 8f  vgatherdps zmm0{k3},DWORD PTR [rdi+zmm1*4]
 402772: 48 8d 3c 06           lea    rdi,[rsi+rax*1]
 402776: 4c 01 d8              add    rax,r11
 402779: 62 f1 7c 48 59 c2     vmulps zmm0,zmm0,zmm2
 40277f: 62 f2 7d 4c a2 04 8f  vscatterdps DWORD PTR [rdi+zmm1*4]{k4},zmm0
...
```

3.3.3 Kernel 3: 3 Element Stencil

```
0000000000402a00 <kernels_ThreeStencilF32>:
...
 402ab8: 62 f2 7d 4b 92 34 98  vgatherdps zmm6{k3},DWORD PTR [rax+zmm3*4]
 402abf: 62 f2 7d 4a 92 2c 9a  vgatherdps zmm5{k2},DWORD PTR [rdx+zmm3*4]
 402ac6: 48 01 f8              add    rax,rdi
 402ac9: 4c 01 e2              add    rdx,r12
 402acc: 62 f1 74 48 59 f6     vmulps zmm6,zmm1,zmm6
 402ad2: 62 f2 7d 4c 92 24 98  vgatherdps zmm4{k4},DWORD PTR [rax+zmm3*4]
 402ad9: 62 f2 4d 48 98 e8     vfmadd132ps zmm5,zmm6,zmm0
 402adf: 62 f2 55 48 98 e2     vfmadd132ps zmm4,zmm5,zmm2
 402ae5: 62 f2 7d 4d a2 24 9e  vscatterdps DWORD PTR [rsi+zmm3*4]{k5},zmm4
...
```

All three kernels utilize AVX512 to compute values with the most parallelism possible, using gather and scatter instructions to account for data striding, and vector masking to account for varying tail sizes.

3.4 Verifying Correctness

To verify correctness of the serial kernels I apply standard formal verification techniques. The kernels are each short enough where formal verification techniques of establishing preconditions and determining the resulting postconditions are sufficient to prove correctness of the kernel.

3.4.1 SAXPY

```
1  // Precondition: a, x, y are each arrays of 'len' number of T elements
2  // Precondition: Type T is either a float or double
3  KERNEL_INLINE void saxpy_impl_##SUFFIX(const T *restrict a,
4  const T *restrict x, T *restrict y,
5  size_t len, size_t stride)
           {
6
7      for (size_t i = 0; i < len; i++) // Loop Terminates since len is a finite
           integer
8
9      y[i * stride] += a[i * stride] * x[i * stride];
10
11   } // Postcondition: y[i * stride ] = a[i * stride] * x[i * stride] + y[i *
       stride] for all i < len
```

Listing 1: SAXPY kernel (C)

3.4.2 Vector Multiply

```
1  // Precondition: x, y, z are each arrays of 'len' number of T elements
2  // Precondition: Type T is either a float or double
3  KERNEL_INLINE void multiply_impl_##SUFFIX(const T *restrict x,
4  const T *restrict y,
5  T *restrict z, size_t len,
6  size_t stride) {
7      for (size_t i = 0; i < len; i++) // Loop Terminates since len is a finite
           integer
8
9      z[i * stride] = x[i * stride] * y[i * stride];
10
11   }
12 // Postcondition: z[i * stride ] = x[i * stride] *  y[i * stride] for all i < len
```

Listing 2: Vector Elementwise Multiplication kernel (C)

3.4.3 Three Stencil

```
1  // Precondition: x, y are each arrays of 'len' number of T elements
2  // Precondition: Type T is either a float or double
3  KERNEL_INLINE void stencil_impl_##SUFFIX(const T *restrict x, T *restrict y,
4  T a, T b, T c, size_t len,
5  size_t stride) {
```

```
6       for (size_t i = 1; i < len - 1; i++) // Loop Terminates since len is a finite
            integer
7
8       y[i * stride] = a * x[(i - 1) * stride] + b * x[i * stride] + c * x[(i + 1) *
            stride];
9
10    }
11
12  // Postcondition: y[i * stride] = a * x[(i - 1) * stride] + b * x[i * stride] + c
        * x[(i + 1) * stride]; for all i < len
```

Listing 3: Three Element Stencil Kernel (C)

4 Measurement Discipline

4.1 Experimental Setup

- **Hardware:** CPU: x86_64 AMD Ryzen AI 7 350 w/ Radeon 860M (SIMD Support: SSE, SSE2, AVX, AVX2, AVX-512), Memory: 32GB LPDDR5 @ 5600MT/s
- **OS / Kernel:** Pop_OS! 24.04 Running Linux 7.0.11-76070011-generic
- **Compiler & Flags:** GCC 16.2.0, Flags: -O2 -march=znver4 -ftree-vectorize (disabled for serial experiments)
- **Isolation techniques:** Ran on CPU2 with 100% affinity, and -20 niceness (ensure no context switching)
- **Trials per condition:** The kernels were run in a loop for 100ms and then run 30 times then reporting the median execution time.
- **Timer / Instrumentation:** clock_gettime(CLOCK_MONOTONIC)
- **CPU Governor:** The CPU Governor was set to 'Performance Mode' on all core with a maximum frequency of 2000MHz

4.2 Statistical Treatment

To ensure that the AVX execution units are warmed up and able to run at their fastest clock. To achieve this, the kernels are run in loops for 100ms without recording performance data before running the measured runs. Then to avoid variation from different processes running on the computer, the kernels are ran 30 times, measuring the performance on each run. Then the median execution time is reported and used in the plots and charts.

5 Results

5.1 Summary Figure

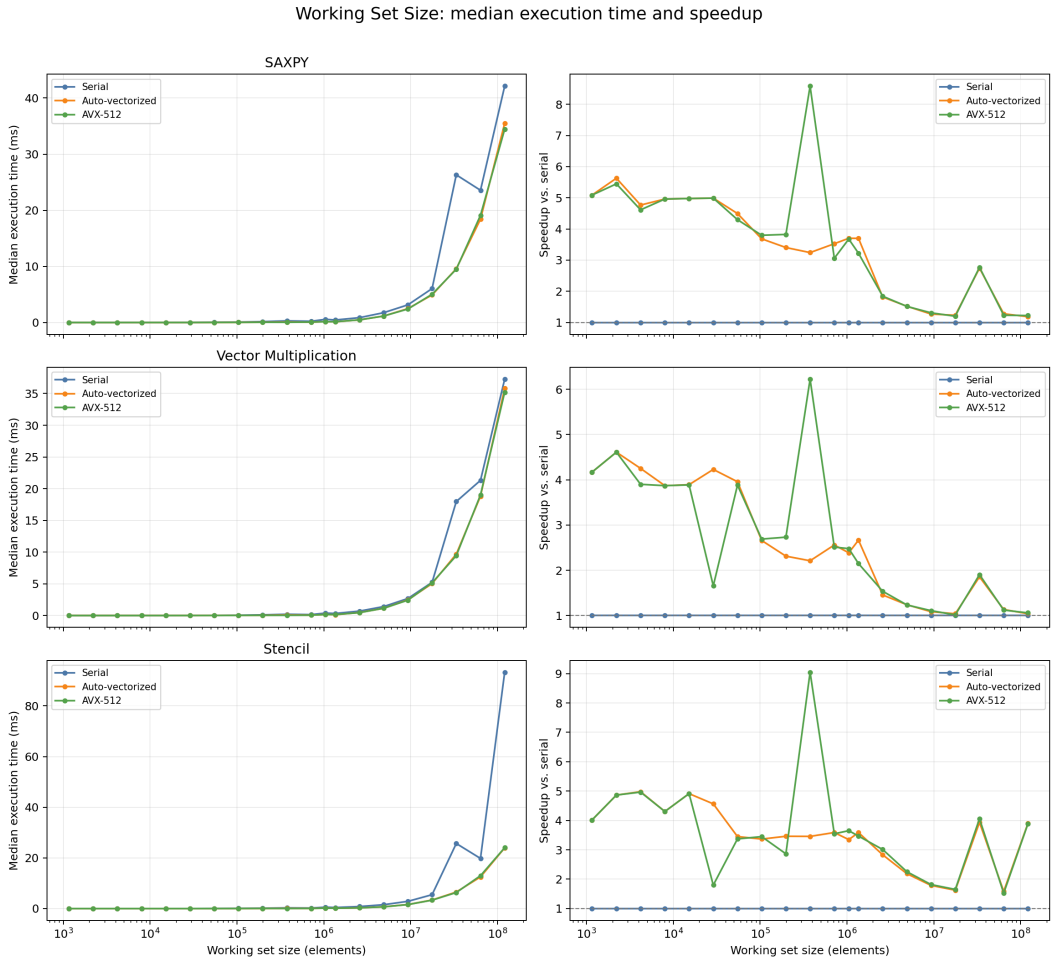


Figure 1: The median execution time and speedup from each implementation as working set size changes. When the working set becomes large enough to sit in DRAM, the bottleneck becomes the time required to fetch data from DRAM rather than time to compute data, for this reason, the speedup decreases significantly as working set size increases.

5.2 Secondary Figures

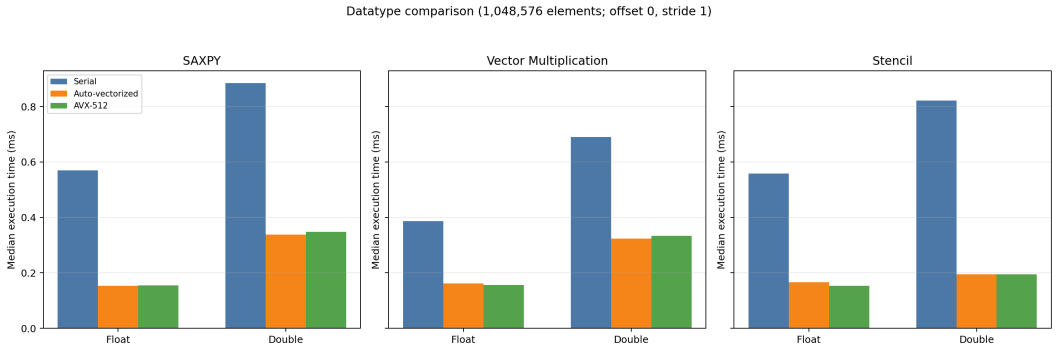


Figure 2: The median execution time for each kernel with a fixed 2^{20} element working set size, stride of 1, and 0 tail elements, only changing the datatype that is used for execution. For SAXPY and Vector Multiplication using the double datatype with vectoriziation cut performance by approximately one-half. However, for the three-stencil switching to the double datatype had a much smaller effect on median execution time when ran with vectoriziation.

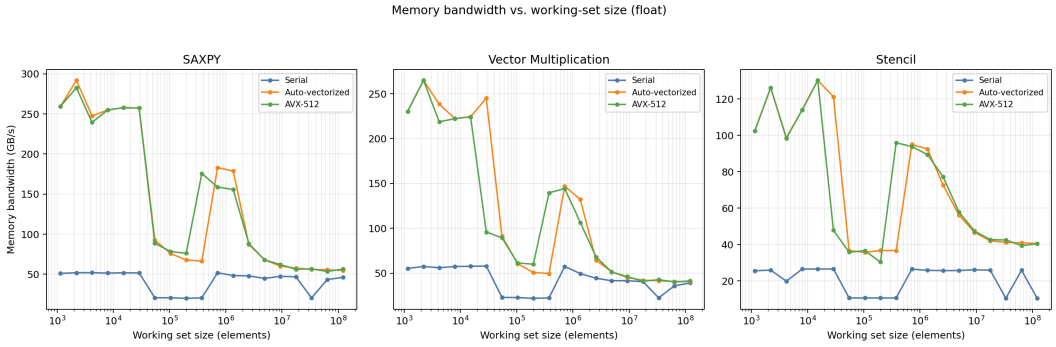


Figure 3: The estimated memory bandwidth for each kernel and execution model as working set size changes. The peak bandwidth (approx. 275GB/s) was attained when data was fully resident in L1 cache. As data moved up the memory hierarchy the bandwidth decreased to approximately 60GB/s.

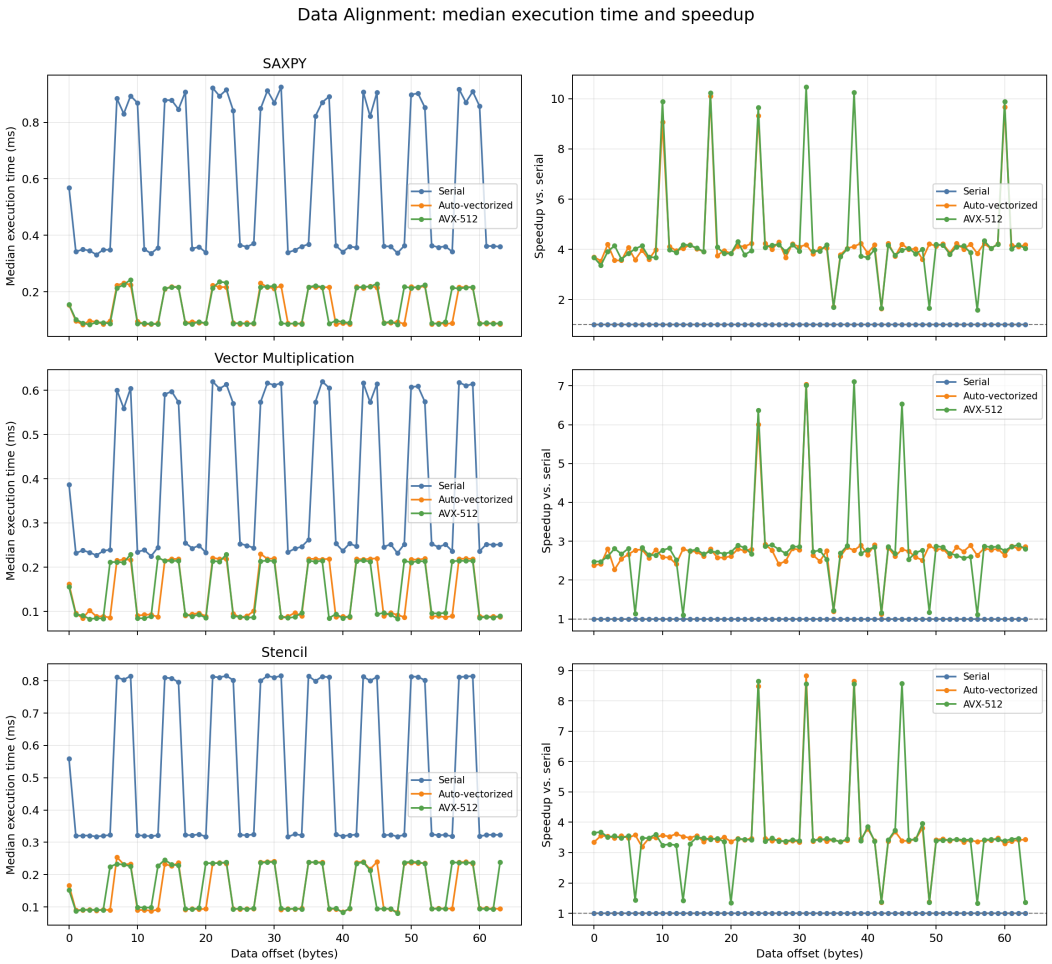


Figure 4: The median execution time for each kernel with a fixed 2^{20} element working set size, stride of 1, and 0 tail elements, only changing the alignment for the base pointer. For this working set size, the vectorized kernels are siginificantly faster than the serial kernels, however having the pointer not aligned to 4-8 Byte offset does significantly impact performance.

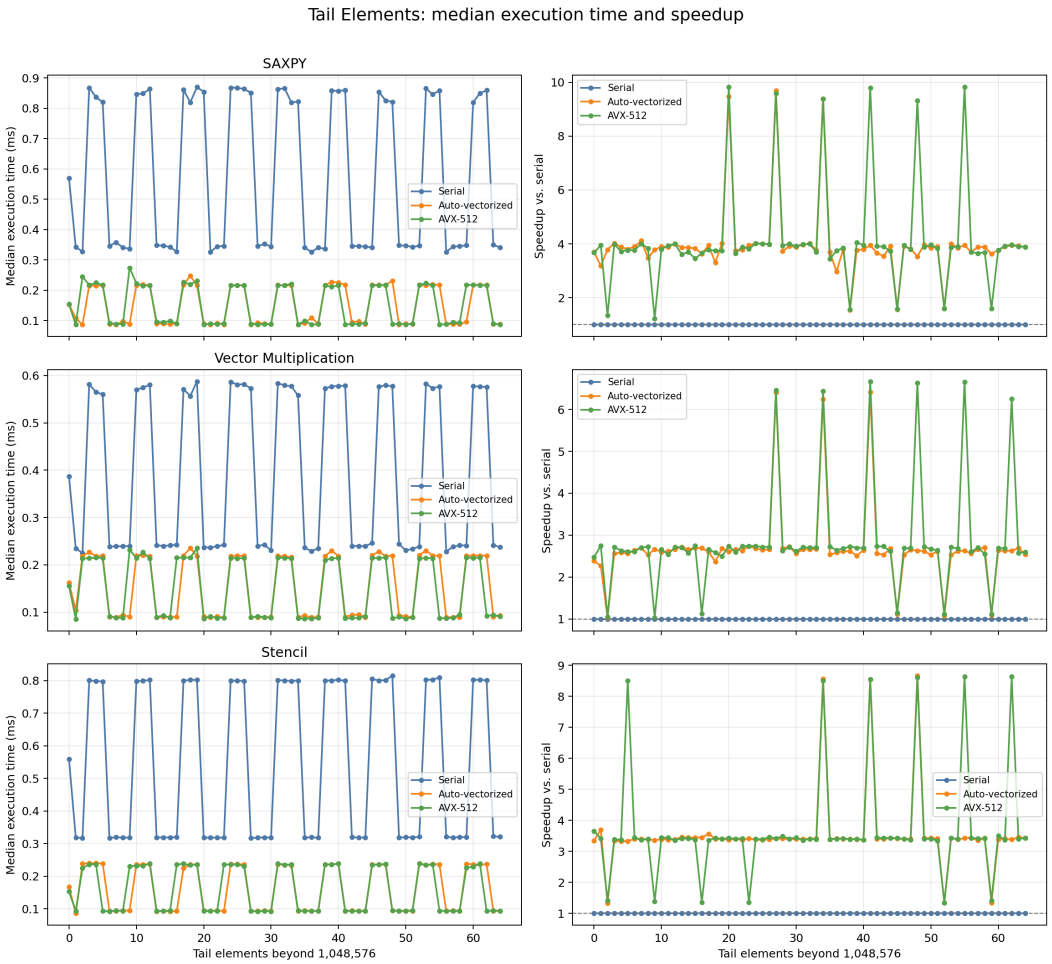


Figure 5: The median execution time for each kernel with a fixed 2^{20} element working set size, stride of 1, and no base pointer offset, only changing the number of tail elements. Since I am using AVX register masking to handle the number of tail elements, there is still a significant speedup on handling tail elements. There is a significant slowdown when the last vector has certain numbers of tail elements. However, since this also appears in the serial kernels, I think there may be some confounding results there.

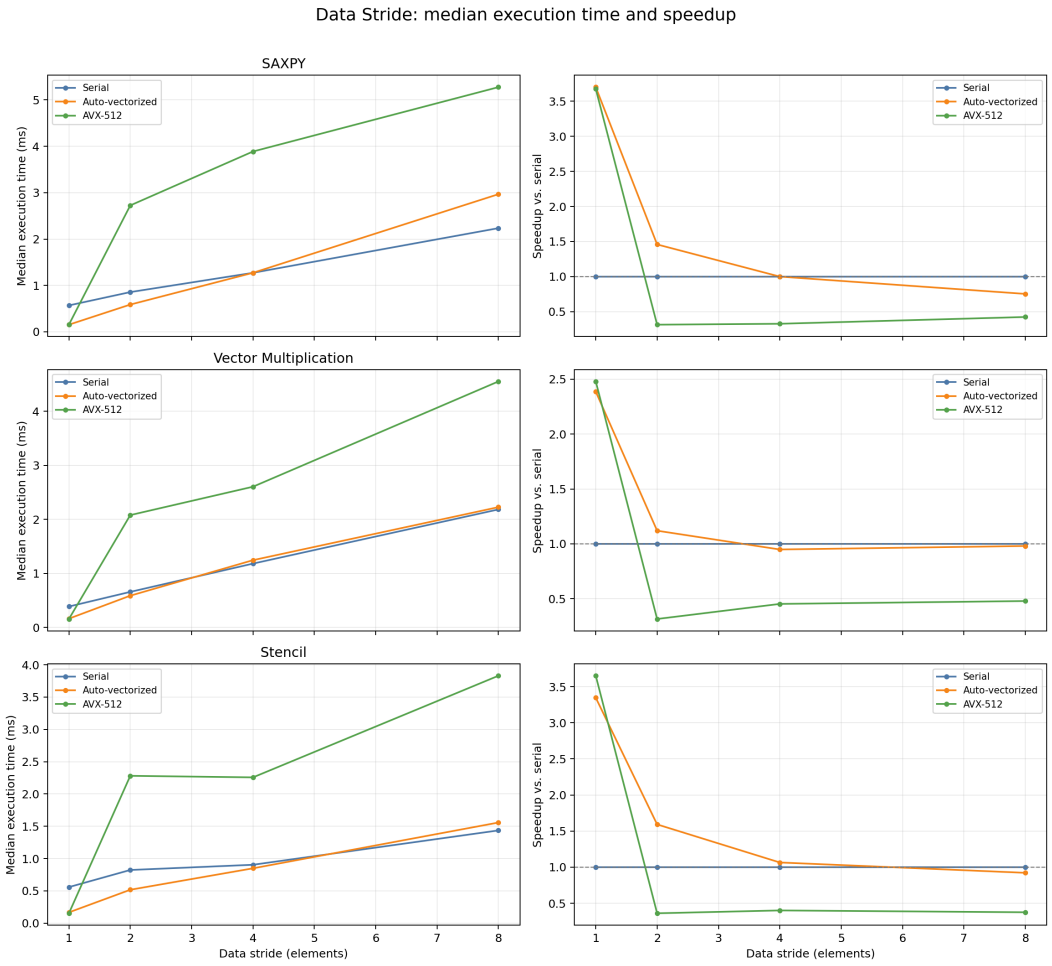


Figure 6: The median execution time for each kernel with a fixed 2^{20} element working set size and no base pointer offset, only changing the stride of the data. Having the data strided at all, made vectoriziation much slower, sometimes even slower than the serial implementation in the case of my own AVX512 implementations. This demonstrates how important contiguous data is for vectorized operations.

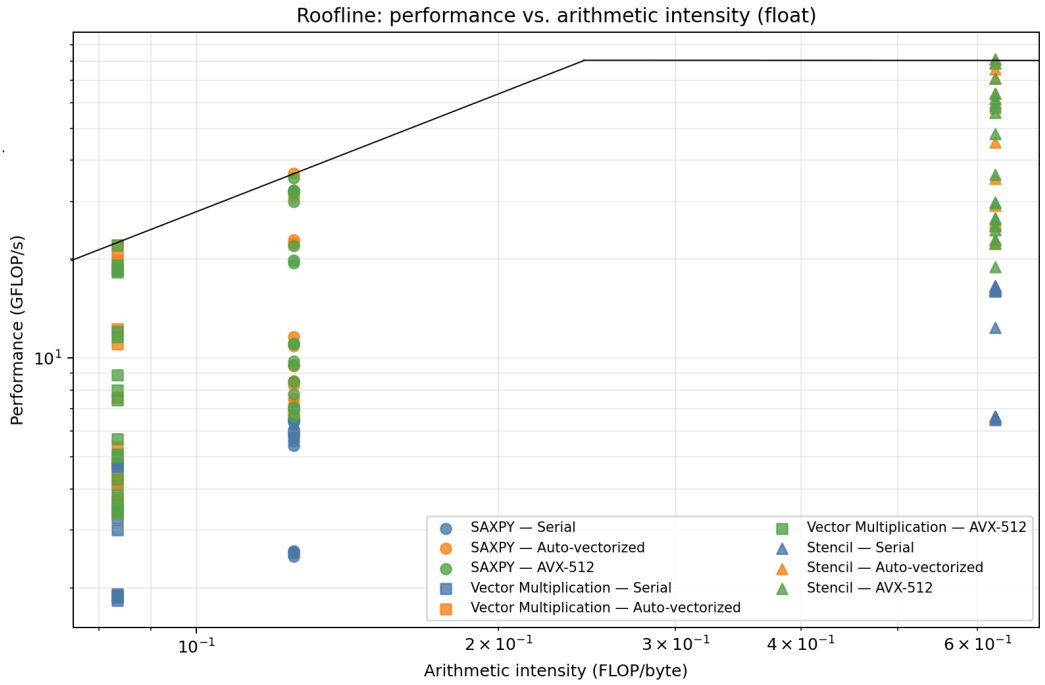


Figure 7: Roofline Plot showing the memory bandwidth vs compute capacity for my computer. The peak computational performance I observed was on the three-element stencil, which had a peak of approximately 80 GFLOP/s. This was attained with the smallest working set size, where all data was resident in L1D cache.

6 Negative / Failure Case

Negative / Failure Case

What failed:
Loops that contain data that is strided did not see any benefit from SIMD vectoriziation. I expected that strided data would be slower than packed data but that SIMD would still provide some speedup. However, in most cases the strided data completely eliminated any speedup from using SIMD. For my handwritten AVX512 code, it was significantly slower than the serial execution.

Evidence:
Figure 6 shows that as soon as data stride exceeds the unit stride, that the performance benefits from vectoriziation disappear or reverse making serial execution faster.

Root cause (if known):
In my handwritten AVX512 implementation, I use 'VGATHER' and 'VSCATTER' which can each take upto 13 or 17 cycles resepectively. Whereas the alligned load only takes 1 cycle and the Fused-Multiply-Add operation also only takes 1 cycle. Gathering and scattering data across strided regions of memory ends up taking most the execution time. [1]

7 Honest Anomalies

Honest Anomalies

Anomaly 1:

As shown in figure 3 the bandwidth falls off a cliff when the working set size becomes greater than 2^{15} elements. Then picks back up as the number of elements grows to 2^{19} elements. This would imply that L3 cache is faster than L2 cache, which is not correct. The exact reason for this is unknown because very few things were running on the other cores.

Anomaly 2:

When changing the number of tail elements to process, as indicated by figure 5 both the serial and the vectorized execution time changed. I wouldn't expect that the serial execution time would change with the number of tail elements.

8 Discussion

Overall my hypotheses held up pretty well. Vectoriziation did introduce a significant speedup, the specific speedup did depend on the data alignment, tailing element size, and working set size. For small working sets, my prediction for speed was somewhat correct, creating speed-ups of around 5x for FP32, whereas I predicted a much greater speedup. My handwritten AVX512 code was not that much faster than what the compiler could create, and in certain cases like strided data significantly slower than even serial execution.

My hypothesis regarding data alignment was also mostly correct, data alignment does have a significant impact on the performance of vectorized code, the actual impact though is less predictable than I would have thought and the vectorized execution is more resilient to misaligned data than I thought it would be.

9 Conclusion

Overall, vectoriziation is a very strong potential compiler optimization for data that is contiguous in memory and is aligned to the 64 byte cache line. Although my handwritten AVX512 vectoriziation was slightly faster than the compiler in certain circumtstances, the compliler produced vectorized code that better handled more general cases. The compiler also didn't spend an entire day trying to write AVX512 code. For this reason, programmers should focus on designing data structures and functions that keep data in contiguous memory and give the compiler as many opportunities to automatically vectorize the code.

A AI Usage

AI helped me write the code required for the experiment, it helped develop scripts for running all the required experiments, assisted in developing python code to create plots and figures. Lastly, I utilized AI in developing the experimental design and troubleshooting anomalies in the data.

1. Generated LaTeX template for the report: https://claude.ai/share/2757c2c9-0ce7-42d1-8546-33ad7abf8f7a

2. Generated the code for the random float data generator ‘./src/data-gen/main.c‘: https://claude.ai/share/68a2ba0a-4a17-439e-b7ba-caf666aabab9
3. Generated the verifier for verifying that two data outputs are the same ‘./src/verifier/main.c‘: https://claude.ai/share/68a2ba0a-4a17-439e-b7ba-caf666aabab9
4. Helped me with understanding the different features of the AVX instruction set: https://claude.ai/share/d8f4d5b0-6c86-444e-be84-7cd45c2d1c63
5. Helped me review my kernel implementations: https://claude.ai/share/2e4bc6ff-4500-42c9-a0de-b693232d2634
6. Updated my implementation to best utilize compiler optimizations: https://claude.ai/share/0ca2f2f2-e344-4c1b-9144-afdf8a9730e5
7. Helped me troubleshoot a roofline plot that didn’t make sense: https://claude.ai/share/49f7c338-1eb1-4fae-9c43-98317c09c256

I also made extensive use of OpenCode (an open source coding agent) to develop scripts, experiment code, and plotting code. I reveiewed all output for correctness and accuracy. Making minor fixes where applicable. Transcripts for the OpenCode sessions are available in `./OpenCode`

B Additional Data / Raw Logs

Full disasemblies can be found in `./disassembly/`. The full CSV data can be found in ‘`./results/data.csv`’. Plotting code can be found in `./analysis`.

References

[1] A. Fog, “Introduction 4. instruction tables,” Tech. Rep., 9 2025, accessed 2026-10-22. [Online]. Available: https://www.agner.org/optimize/instruction_tables.pdf